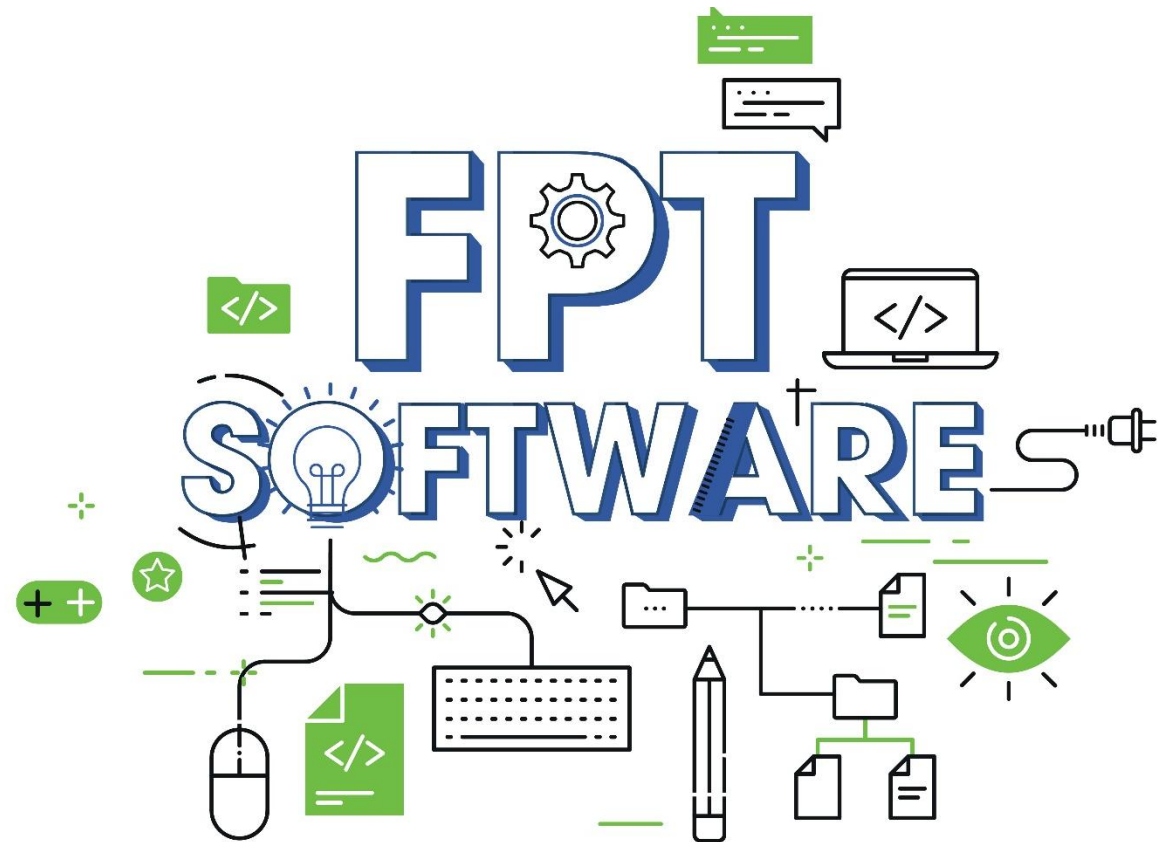


Basic Android

Android Overview



Agenda

1. Introduction to Android Architecture

2. Introduction to Android automotive

3. Prepare environment

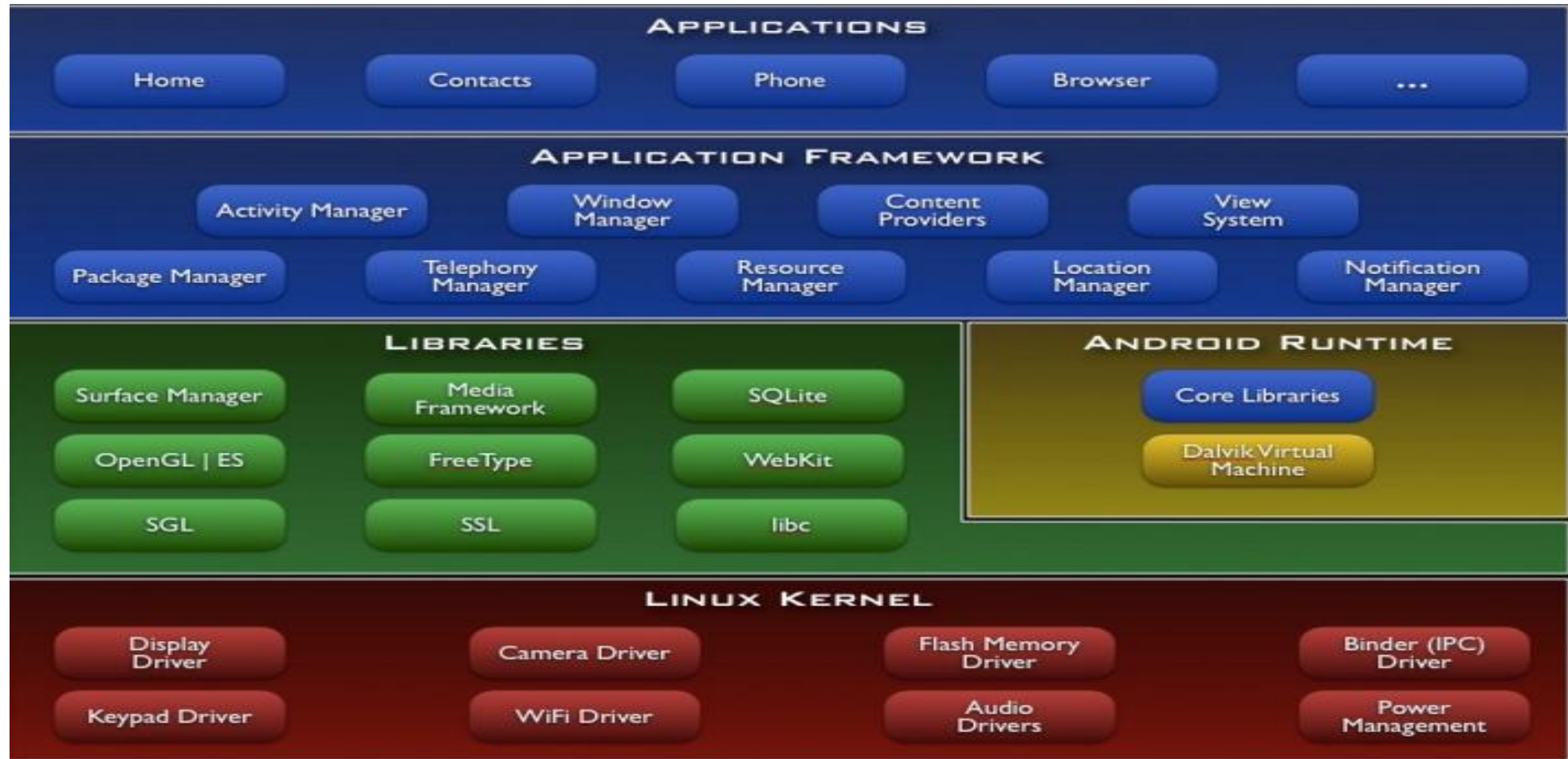
4. Create and Run an Android app

5. USB Debugging



Introduction to Android Architecture

Architecture Android



Android S/W Stack - Application

- Android provides a set of core applications:
 - ✓ Email Client
 - ✓ SMS Program
 - ✓ Calendar
 - ✓ Maps
 - ✓ Browser
 - ✓ Contacts
 - ✓ Etc
- All applications are written using the Java language.



Android S/W Stack – App Framework

- Enabling and simplifying the reuse of components
 - ✓ Developers have full access to the same framework APIs used by the core applications.
 - ✓ Users are allowed to replace components.



Android S/W Stack – App Framework(Cont)

Feature	Role
View System	Used to build an application, including lists, grids, text boxes, buttons, and embedded web browser
Content Provider	Enabling applications to access data from other applications or to share their own data
Resource Manager	Providing access to non-code resources (localized strings, graphics, and layout files)
Notification Manager	Enabling all applications to display customer alerts in the status bar
Activity Manager	Managing the lifecycle of applications and providing a common navigation backstack

Android S/W Stack – App Framework

- Enabling and simplifying the reuse of components
 - ✓ Developers have full access to the same framework APIs used by the core applications.
 - ✓ Users are allowed to replace components.



Android S/W Stack – Libraries

- Including a set of C/C++ libraries used by components of the Android system
- Exposed to developers through the Android application framework



Android S/W Stack – Runtime

■ Core Libraries

- Providing most of the functionality available in the core libraries of the Java language
- APIs
 - Data Structures
 - Utilities
 - File Access
 - Network Access
 - Graphics
 - Etc



Android S/W Stack – Linux Kernel

- Relying on Linux Kernel 2.6 for core system services
 - Memory and Process Management
 - Network Stack
 - Driver Model
 - Security
- Providing an abstraction layer between the H/W and the rest of the S/W stack



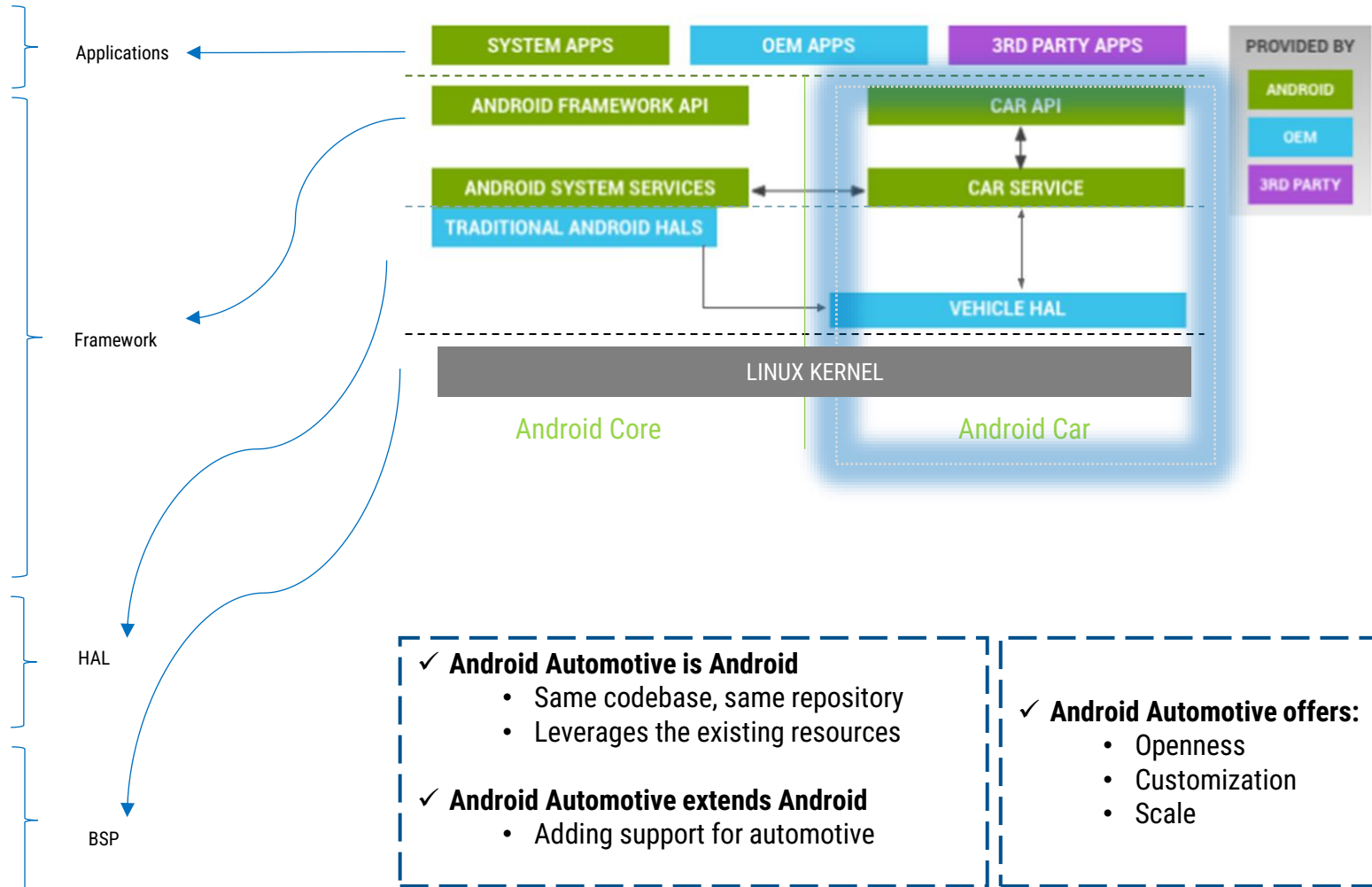
Introduction to Android automotive

Introduction to Android automotive

Android System architecture



Android Automotive architecture



Prepare environment

- Phone
- Android Studio ([Download Android Studio & App Tools - Android Developers](#))



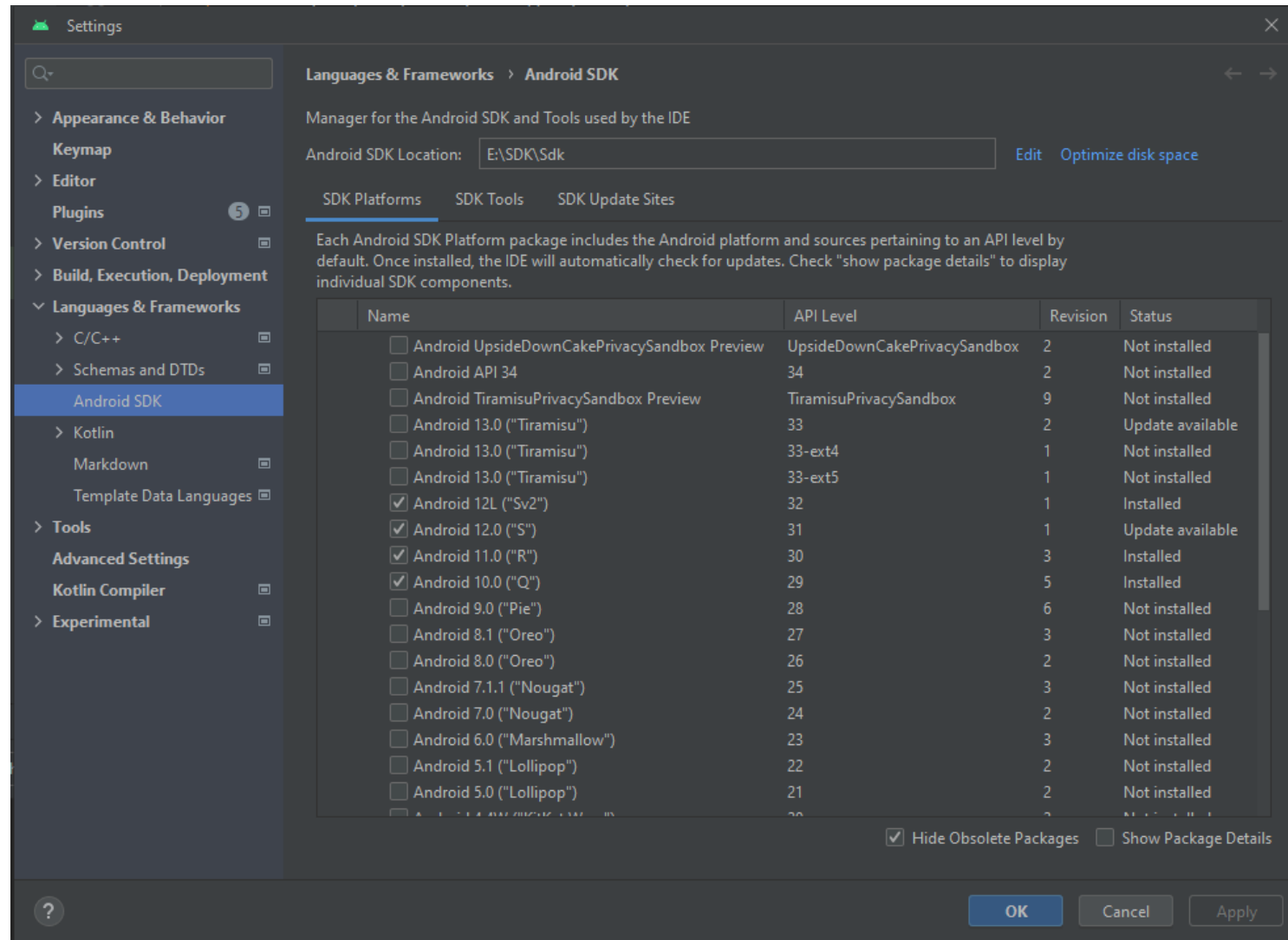
Android SDK

- Once installed open the SDK Manager
- Install the desired packages
- Create an Android Virtual Device (AVD)



Android SDK

SDK Manager





Virtual Device Configuration

Select Hardware

Choose a device definition

Q

Category	Name	Play Store	Size	Resolution	Density
Phone	Resizable (Experimen...		6.0"	1080x2340	420dpi
Tablet	Pixel XL		5.5"	1440x2560	560dpi
Wear OS	Pixel 7 Pro	▶	6.71"	1440x3120	560dpi
Desktop	Pixel 7	▶	6.31"	1080x2400	420dpi
TV	Pixel 6a	▶	6.13"	1080x2400	420dpi
Automotive	Pixel 6 Pro		6.7"	1440x3120	560dpi
	Pixel 6		6.4"	1080x2400	420dpi

New Hardware Profile

Import Hardware Profiles

Pixel 2

1080px

5.0"

1920px

Size: large

Ratio: long

Density: 420dpi

Clone Device...

?

Previous

Next

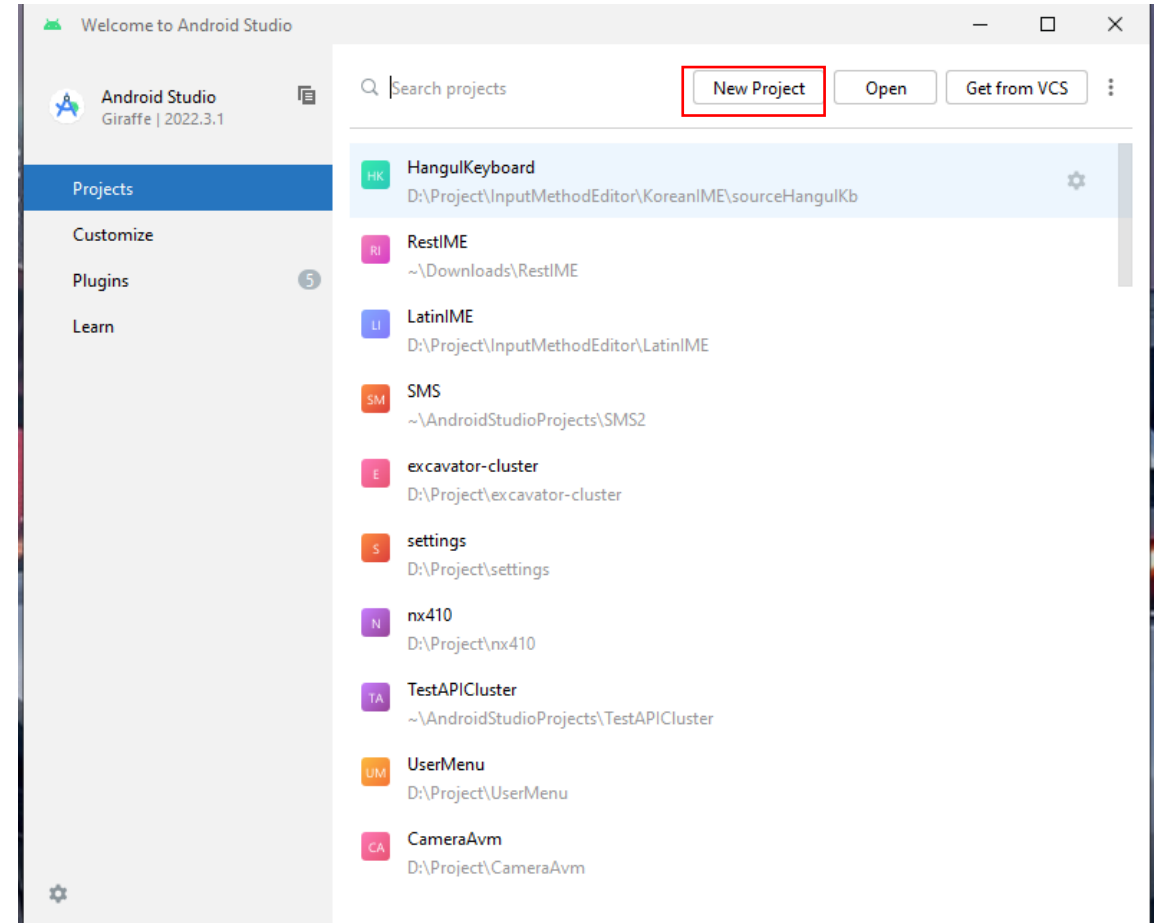
Cancel

Finish

Create and Run an Android app

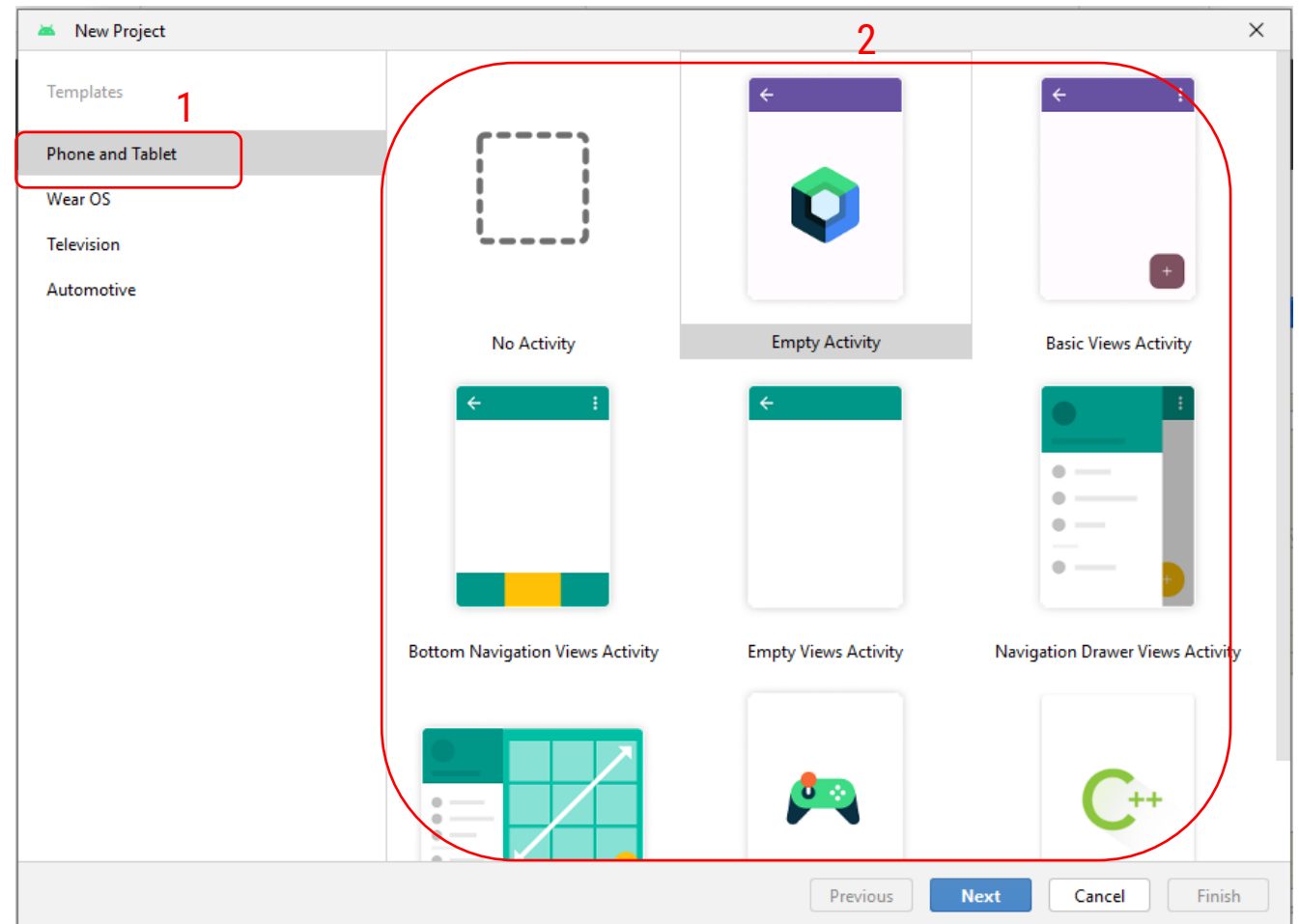
Creating a Project

- Run Android Studio> Click to 'New Project'



Creating a Project (Cont)

1. Select 'Phone and Tablet'
2. select any template



Creating a Project (Cont)

■ Fill up:

1. Name: Name of application
2. Package name: format com.xxx.xxx
3. Save location
4. Minimum SDK
5. Build config.

New Project

Empty Activity

Create a new empty activity with Jetpack Compose

Name: My Application

Package name: com.example.myapplication

Save location: C:\Users\NamHV4\AndroidStudioProjects\MyApplication3

Minimum SDK: API 24 ("Nougat"; Android 7.0)

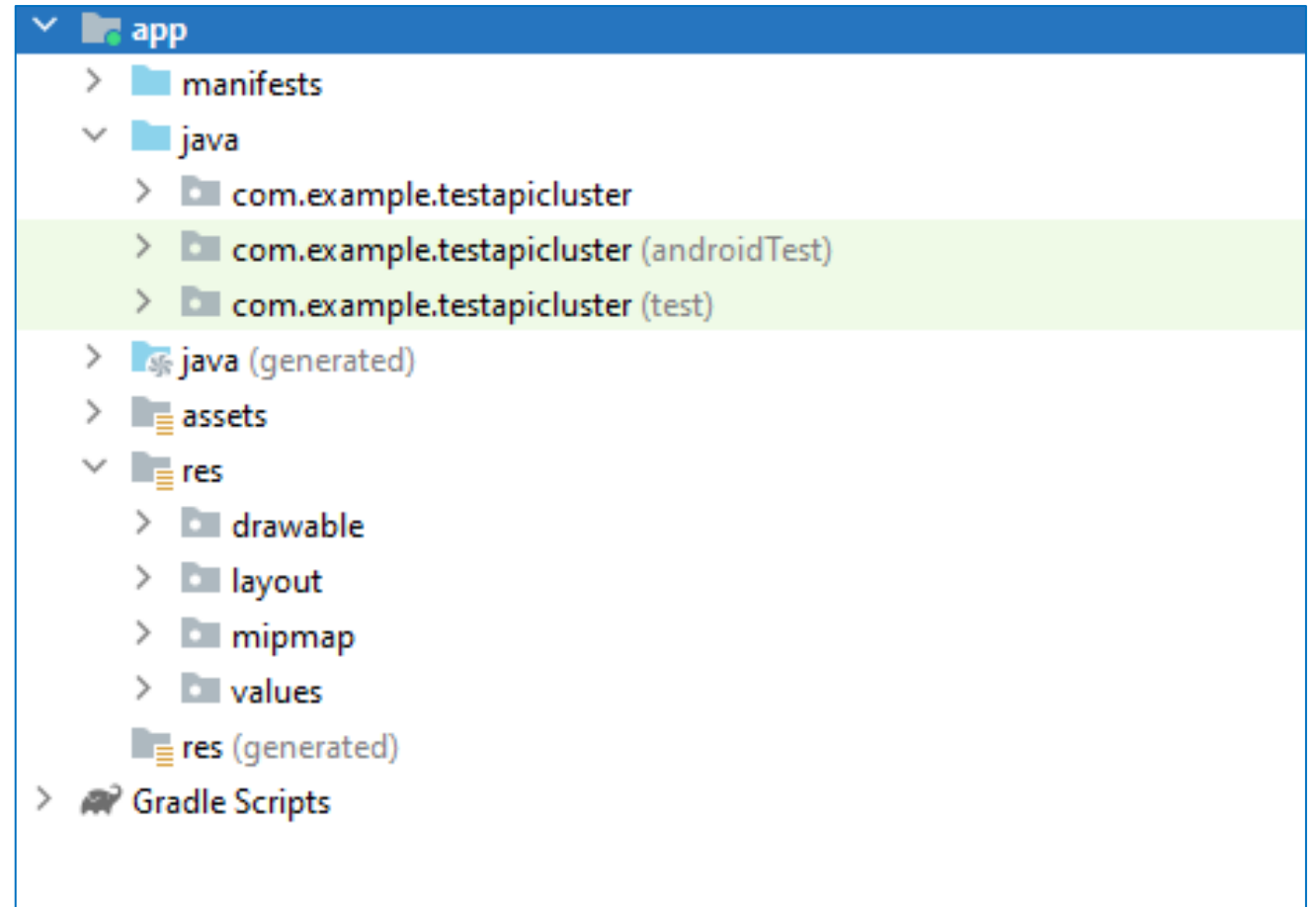
Information: Your app will run on approximately 95.4% of devices. [Help me choose](#)

Build configuration language: Kotlin DSL (build.gradle.kts) [Recommended]

Buttons: Previous, Next, Cancel, Finish

Project Components

- App
 - Manifests
 - Java
 - Java (generated)
 - Assets
 - res
 - ✓ Drawable
 - ✓ Layout
 - ✓ Mipmap
 - ✓ values
- Gradle: Config build, add library



Manifest File

- Contains characteristics about your application
- When have more than one Activity in app, NEED to specify it in manifest file
 - ✓ Go to graphical view of the manifest file
 - ✓ Add an Activity in the bottom right
 - ✓ Browse for the name of the activity
- Need to specify Services and other components too
- Also important to define permissions and external libraries, like Google Maps API

- Auto-generated: you shouldn't edit it
- Contains IDs of the project resources
- Enforces good software engineering
- Use findViewById and Resources object to get access to the resources

✓ Ex. Button b = (Button)findViewById(R.id.button1)

✓ Ex. getResources().getString(R.string.hello));

USB Debugging

USB Debugging

- Should be enabled on phone to use developer features
- In the main apps screen select Settings -> Applications -> Development -> USB debugging (it needs to be checked)

Android Debug Bridge

- Used for a wide variety of developer tasks
 - ✓ Read from the log file
 - ✓ Show what android devices are available
 - ✓ Install android applications (.apk files)
- In the 'platform-tools' directory of the main android sdk directory
 - ✓ Recommend putting this directory and the 'tools' directory on the system path
- adb.exe

- Instead of using traditional `System.out.println`, use the Log class
 - ✓ Imported with `android.util.Log`
 - ✓ Multiple types of output (debug, warning, error, ...)
 - ✓ `Log.d(<tag>, <string>)`
- Can be read using logcat.
 - ✓ Print out the whole log, which auto-updates
 - `adb logcat`
 - ✓ Erase log
 - `adb logcat -c`
 - ✓ Filter output via tags
 - `adb logcat <tag>:<msg type> *:S`
 - can have multiple `<tag>:<msg type>` filters
 - `<msg type>` corresponds to debug, warning, error, etc.
 - If use `Log.d()`, then `<msg type> = D`
- Reference
 - ✓ <http://developer.android.com/guide/developing/debugging/debugging-log.html>

References

- This lesson is a brief overview of some major concepts
- Developer's Guide
 - ✓ <http://developer.android.com/guide/index.html>
- API Reference
 - ✓ <http://developer.android.com/reference/packages.html>

THANK YOU!

